Electrónica Digital 1

TRABAJO PRÁCTICO FINAL

Escuela de Ciencia y Tecnología

Segundo Cuatrimestre, 2026

Alumnos:

Romero Marchione Luz: lromeromarchione@estudiantes.unsam.edu.ar

Roca Garza Sebastián: srocagarza@estudiantes.unsam.edu.ar

Profesores:

Miguel Angel Sagreras

Nicolas Alvarez

OBJETIVOS

El presente trabajo práctico tuvo como objetivo el diseño e implementación en FPGA de un voltímetro digital de tres canales. El sistema desarrollado está compuesto por conversores A/D Sigma-Delta, bloques de procesamiento de medición y una salida VGA que permite visualizar simultáneamente los valores obtenidos.

ESPECIFICACIONES

Para el diseño del voltímetro se empleó el lenguaje de descripción de hardware VHDL, utilizando ModelSim Altera Starter Edition 10.1d para la simulación de los bloques. Posteriormente, se utilizó Quartus II como herramienta de síntesis e implementación del diseño en la FPGA.
La FPGA utilizada corresponde a una placa basada en el dispositivo EP2C5, perteneciente a la familia Cyclone II. Para la conexión del sistema se utilizaron señales de entrada asociadas a los tres canales de medición, señales de salida para la realimentación de los conversores A/D Sigma-Delta y las salidas necesarias para la interfaz VGA. Las asignaciones principales de pines indicadas por la cátedra son las siguientes:

Puerto	Pin
clk_i	17
rst_i	144
data_volt_in_0_i	89
data_volt_out_0_o	87
data_volt_in_1_i	75
data_volt_out_1_o	73
data_volt_in_2_i	101
data_volt_out_2_o	74
hs_o	79
vs_o	99
red_o	96
grn_o	92
blu_o	86

Tabla 1: Asignación de pines físicos utilizados en la FPGA

DIAGRAMA DE BLOQUES

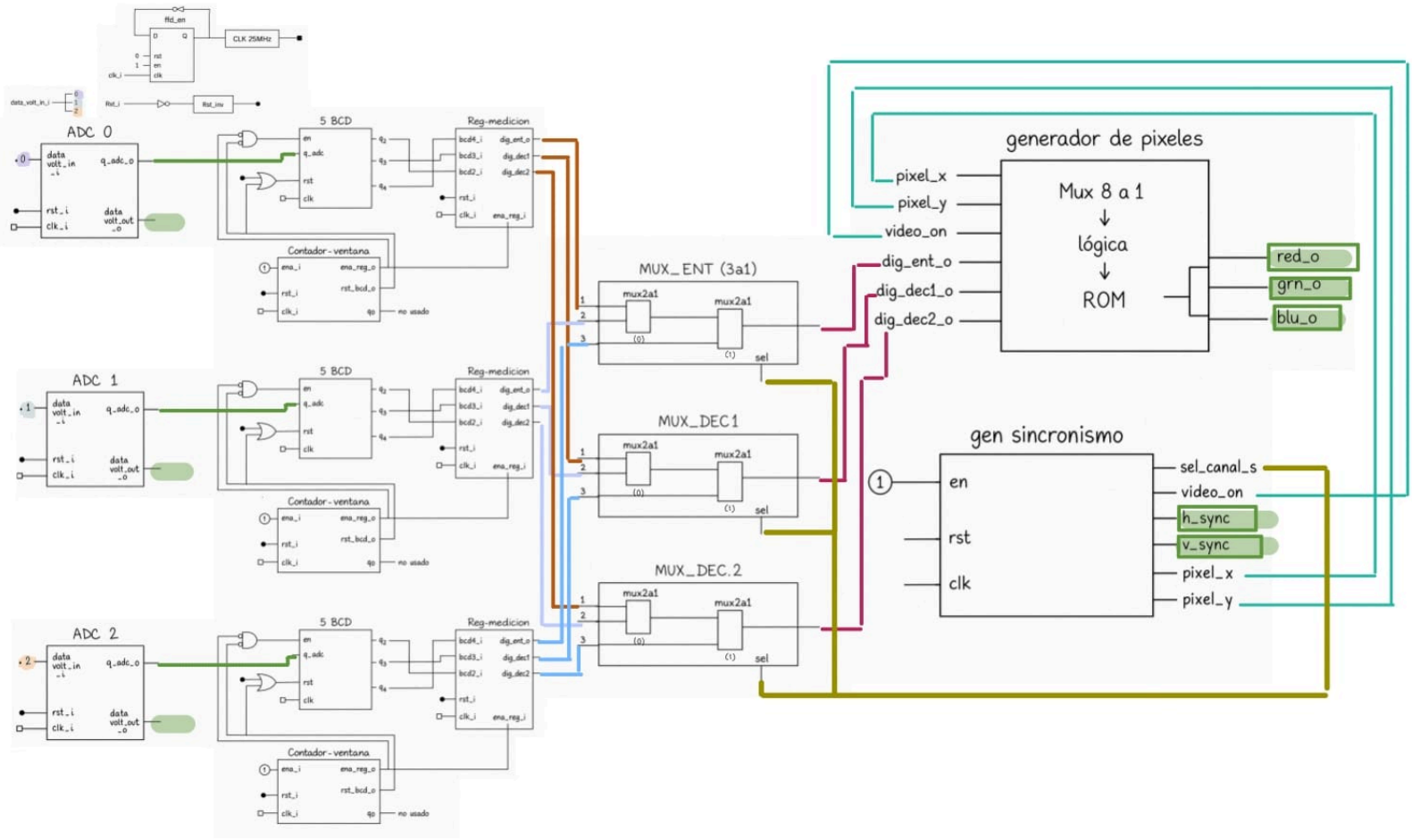


Fig.1 *Diagrama de Bloques.*

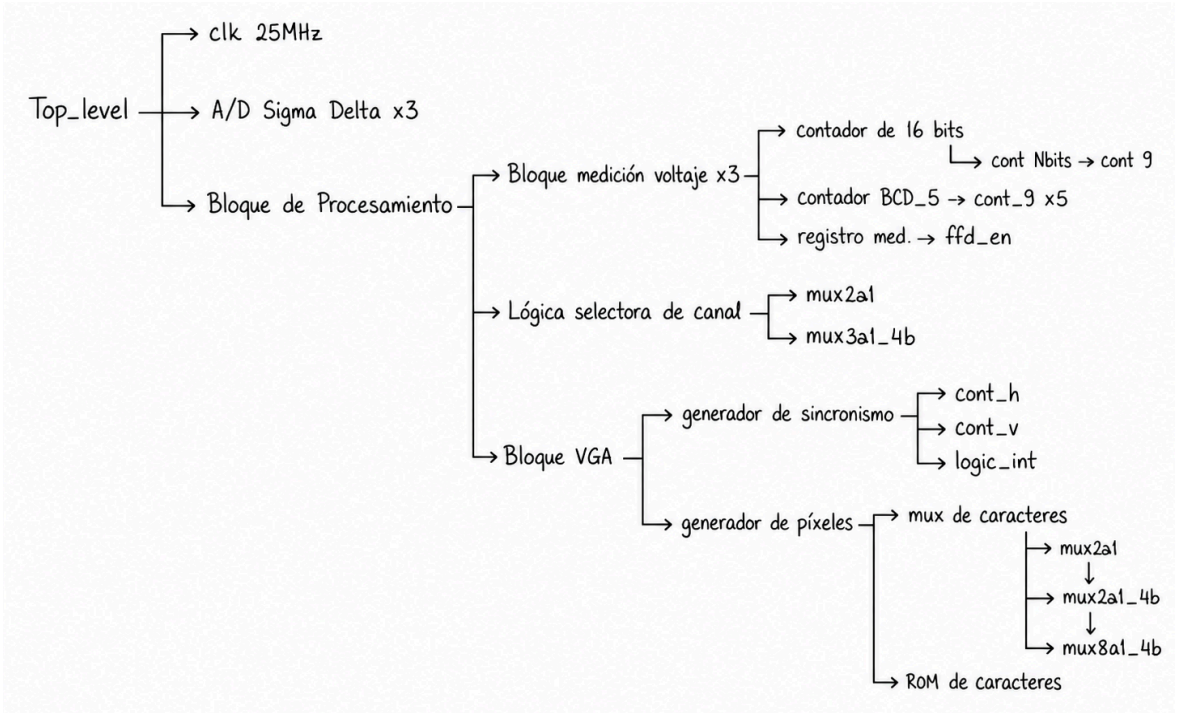


Fig.2 *.Diagrama jerárquico de bloques y componentes .*

DESARROLLO DE LOS COMPONENTES

1. TOP LEVEL

El diseño completo se organiza a partir del top_level, que funciona como entidad principal del sistema. Dentro de este bloque se integran los componentes generales del voltímetro: los tres conversores A/D Sigma-Delta, el bloque de procesamiento y la etapa de generación del reloj de 25 MHz.

1.1 CLOCK MODIFICADO

La placa utilizada posee un reloj principal de 50 MHz, mientras que el sistema VGA requiere un reloj de aproximadamente 25 MHz. Por este motivo, en top_level se implementó un divisor de frecuencia utilizando un flip-flop con habilitación (ffd_en).

1.1.1 Flip-flop D con habilitación (ffd_en)

El flip-flop D es un elemento de memoria que almacena un bit de información. En cada flanco ascendente del reloj (clk), copia el valor de su entrada D a la salida Q, siempre que la señal de habilitación (ena) sea 1. Si ena = 0, conserva su estado anterior. Además, cuenta con un reset sincrónico (srst) que permite llevar la salida a 0 en el siguiente flanco ascendente, independientemente del valor de ena.

Para dividir la frecuencia, se conectó la entrada D a la negación de la salida Q del flip-flop, con ena = 1. De esta manera, en cada flanco ascendente del reloj la salida alterna entre 0 y 1. Como se necesitan dos ciclos del reloj original para completar un período de la salida, su frecuencia se reduce a la mitad, obteniendo 25 MHz.

La señal resultante, clk25MHz, se utiliza como reloj común para todos los bloques instanciados en top_level.

1.2 CONVERSOR A/D SIGMA-DELTA

El conversor A/D Sigma-Delta es el bloque encargado de transformar la señal analógica de entrada en una señal digital de un bit. Su funcionamiento se basa en una realimentación: la señal de entrada se conecta al flip-flop D y la salida negada del mismo se devuelve al circuito externo mediante la red RC. De esta manera, la salida del flip-flop genera una secuencia de unos y ceros cuya variación depende del voltaje aplicado.

En el código, este comportamiento se implementa mediante el bloque ff_adc. La señal data_volt_in_i ingresa al flip-flop y, en cada flanco ascendente del reloj, se copia a la salida q_adc_o. A su vez, data_volt_out_o se obtiene como la negación de q_adc_o, cerrando la realimentación del conversor. Como el voltímetro desarrollado posee tres canales, esta misma lógica se instancia tres veces en el top_level, generando tres señales digitales independientes que luego ingresan al bloque de procesamiento.

1.3 BLOQUE DE PROCESAMIENTO

El bloque_procesamiento está formado por tres bloques medicion_voltaje, los multiplexores selectores de canal y el bloque_vga; su función es obtener las mediciones de los tres canales y organizar qué valor se envía a la salida VGA en cada región de la pantalla.

1.3.1 BLOQUE MEDICIÓN_VOLTAJE

El bloque de medición es el encargado de procesar la señal digitalizada proveniente del ADC y convertirla a formato BCD (Decimal Codificado en Binario) estructurando el valor resultante en un dígito entero y dos decimales. Este bloque se compone de tres elementos principales:

1.3.1.1 <u>Contador de 16 bits (ventana de tiempo)</u>

En el ADC Sigma-Delta de 1 bit, el voltaje medido es proporcional a la densidad de unos lógicos que lee la FPGA. Para poder medir esto, el sistema necesita definir un intervalo de tiempo fijo de observación. Acá entra el contador de 16 bits (o ventana de tiempo). Por cuestiones de diseño el contador cuenta hasta 33001 ciclos de reloj. En el siguiente flanco de reloj, el propio contador vuelve a cero, comenzando una nueva ventana de medición.
Su función es establecer la duración exacta de la medición, y además durante su funcionamiento genera dos eventos importantes:

- Habilitación de registros (ena_reg_o): Cuando el contador alcanza el valor 33000 se activa la señal que permite guardar el valor actual de la medición en los registros.
- Reinicio de los BCD (rst_bcd_o): Inmediatamente después al alcanzar el valor 33001 se activa la señal para reiniciar el contador de BCD a cero, preparando el sistema para una nueva medición.

1.3.1.2 <u>Cinco contadores BCD</u>

Este bloque es el encargado de cuantificar el voltaje de entrada mediante cinco contadores de 4 bits conectados en cascada, donde el primer contador (q0) incrementa su cuenta únicamente si recibe un nivel lógico ‘1’ proveniente de la salida del ADC.
Aunque el bloque BCD utiliza cinco dígitos para lograr la resolución necesaria en el conteo de los pulsos generados durante la ventana de tiempo, todo el sistema está diseñado para enviar hacia los registros únicamente los tres dígitos más significativos (q4,q3,q2), descartando los dos menos significativos (q1,q0).

Descartar estos dos últimos dígitos equivale, en la práctica, a realizar una división por 100 sin necesidad de implementar operaciones aritméticas complejas en el hardware. Esto resulta fundamental por la escala de tensión de la placa: como los pines lógicos de la FPGA operan con un voltaje máximo de **3.3 V** (lo que representa el 100% de la escala), si se mide este valor máximo constante, el ADC entregará un '1' ininterrumpidamente y el contador acumulará exactamente **33000** pulsos. Al aplicar la división por 100 descartando los últimos dos dígitos, el valor transferido a los registros es **"330"**.

El diseño lógico del contador decimal cont_9, junto con su tabla de transición, diagrama de estados y mapas de Karnaugh, se presenta en el Anexo A.1.

1.3.1.3 Registros de Medición

Esta entidad se encarga de almacenar y mantener estable el valor medido para que la lógica de la VGA pueda dibujarlo en pantalla, esto debido a que los BCD incrementan y vuelven a cero constantemente su valor en cada medición, si la VGA leyera los datos directamente de ellos se observaría un cambio muy brusco de los números generando como un “parpadeo” constante.

Para solucionar esto, el bloque implementa una etapa de memoria que actúa “fijando” el resultado final mediante tres registros de 4 bits (reg_Nbits), construidos internamente con Flip-Flops tipo D. El funcionamiento se sincroniza con la ventana de tiempo: al llegar el contador a su límite de 33000, emite un pulso de habilitación (ena_reg_i) que ordena a los registros capturar simultáneamente los tres dígitos BCD más significativos de la medición (bcd4_i, bcd3_i, bcd2_i). Estos valores se mantienen constantes e inalterables durante toda la siguiente ventana de medición, mientras los BCD calculan el próximo dato en segundo plano. Estos salen de la entidad como: dig_ent_o para el número entero, dig_dec1_o para el primer decimal y dig_dec2_o para el segundo.

1.3.2 *LÓGICA SELECTORA DE CANAL (multiplexer)*

Luego de procesar los tres canales en paralelo, cada bloque de medición entrega tres dígitos BCD: entero, primer decimal y segundo decimal. Sin embargo, el generador de píxeles recibe un único conjunto de dígitos por vez, por lo que fue necesario implementar una etapa de multiplexado que permita seleccionar la medición correspondiente a cada región de la pantalla.

La selección depende de la posición vertical del barrido. Para ello, gen_sinc genera la señal sel_canal_s a partir de tres bandas verticales, una para cada canal. La lógica utilizada para obtener esta señal se presenta en el Anexo A.3.

Para seleccionar los datos se implementaron tres multiplexores mux3a1_4b, uno para cada dígito de la medición. Estos se construyeron mediante dos multiplexores 2 a 1 de 4 bits conectados en cascada y comparten la misma señal selectora, garantizando que los tres dígitos correspondan siempre al mismo canal. Su implementación y tabla de selección se presentan en el Anexo A.4.

Finalmente, los dígitos seleccionados se envían al bloque VGA para su representación en pantalla.

1.3.3 *BLOQUE VGA*

El bloque_vga integra el gen_sinc y el gen_pixeles en una única entidad encargada de generar la salida de video. Su función principal es conectar las señales de sincronismo, coordenadas y zona visible con la lógica que dibuja los dígitos de la medición, obteniendo finalmente las salidas **hs_o, vs_o, red_o, grn_o y blu_o**. Además, entrega sel_mux_canales, que indica qué banda vertical se está recorriendo y se utiliza dentro del bloque_procesamiento para seleccionar los dígitos del canal correspondiente.

1.3.3.1 Generador de sincronismo

La función principal del generador de sincronismo es generar las coordenadas de dibujo y las señales de temporización requeridas para un correcto barrido del monitor. Para lograr esto se divide en tres partes principales:

- **Contadores de coordenadas**:

. Contador horizontal (cont_h) : se encarga de llevar la cuenta de la coordenada X. Incrementa su cuenta de forma continua en cada ciclo del reloj de píxel. Al completar los 800 ciclos correspondientes a una línea completa, se reinicia a cero y emite un pulso de habilitación hacia el contador vertical.

. Contador vertical (cont_v) : se encarga de llevar la cuenta de la coordenada Y. A diferencia del horizontal, no incrementa con cada ciclo de reloj, sino únicamente cuando el contador horizontal completa una línea entera y genera su pulso de habilitación. Se reinicia al alcanzar las 525 líneas

- **Lógica de intervalos (Sincronismos y Área visible):**

. Área visible: Para la resolución de 640x480, se define como el área donde es válido dibujar. La lógica detecta cuando el pixel X está entre 0 y 639 y cuando el pixel Y está entre 0 y 479. Si ambas coordenadas están dentro del rango valido la señal **video_on** se activa, fuera de este rango la señal se apaga.

. Pulso de sincronismo (h_sync y v_sync): El monitor necesita recibir pulsos de sincronismo negativo (es decir, activos en bajo '0') para saber en qué momento exacto saltar a la siguiente línea (h_sync) o cuándo reiniciar el barrido de la pantalla completamente (v_sync). Estos pulsos de estado bajo se ejecutan en momentos precisos fuera del área visible, justo después de un breve período de espera inactivo (Front Porch), garantizando que los saltos del haz ocurran sin interferir en la imagen proyectada.

Para generar las señales de habilitación de video y sincronismo se implementaron cuatro máquinas de estados de Moore, utilizando el bloque logic_int y flip-flops D. Su diseño lógico y diagramas de estados se presentan en el Anexo A.2.

- **Selección de bandas verticales**

Además de generar las señales de sincronismo, gen_sinc utiliza los bits más significativos del contador vertical para identificar las tres bandas de 128 píxeles de altura destinadas a cada canal. A partir de ellas se genera la señal sel_canal_s, que controla los multiplexores del bloque de procesamiento explicados en el apartado 1.3.2. Las expresiones lógicas y la tabla de selección de bandas se presentan en el Anexo A.3.

1.3.3.1 Generador de píxeles

El bloque gen_pixeles es el encargado de decidir, para cada coordenada de la pantalla, si el píxel debe encenderse o permanecer apagado. Para esto utiliza las coordenadas **pixel_x_i** y **pixel_y_i**, la señal **video_on_i**, los dígitos de la medición y dos bloques internos principales: el mux_char_address y la ROM de caracteres.

1.3.3.1.1 Multiplexor de caracteres (mux_char_address)

El bloque mux_char_address recibe los tres dígitos de la medición (dig_ent_i, dig_dec1_i y dig_dec2_i) y la señal selectora sel_i, proveniente de sel_s. Esta última se genera en gen_pixeles a partir de los bits pixel_x_i(9 downto 7), permitiendo seleccionar un carácter diferente cada 128 píxeles horizontales. Su función es entregar en char_address_o el código del carácter que debe buscarse en la ROM.

Para su implementación se partió de un multiplexor 2 a 1 de 1 bit (mux2a1), a partir del cual se construyó el mux2a1_4b mediante generate, instanciando cuatro multiplexores de 1 bit. Posteriormente, estos se conectaron en cascada para implementar un multiplexor 8 a 1 de 4 bits (mux8a1_4b).

El formato de visualización requiere seleccionar cinco caracteres: dígito entero, punto decimal, dos decimales y unidad V. Por ello, las entradas del multiplexor se asignaron según los códigos definidos en la ROM, combinando los dígitos variables de la medición con códigos fijos para el punto, la letra V y los espacios correspondientes a las posiciones no utilizadas.

Los esquemas de implementación y la tabla de selección de caracteres se presentan en el Anexo A.5.

1.3.3.1.2 ROM de caracteres

La ROM de caracteres contiene la información gráfica de cada símbolo, representado mediante una matriz de 8×8 píxeles. Recibe el código del carácter a través de char_address_s y las coordenadas internas font_row_s y font_col_s, que indican qué fila y columna de la matriz se debe consultar. Como resultado, entrega un bit (rom_out_s) que determina si el píxel correspondiente al carácter debe encenderse.

Para ampliar los caracteres a 128×128 píxeles, se utilizaron los bits (6 downto 4) de las coordenadas horizontal y vertical. De esta forma, cada píxel original de la ROM se representa mediante una región de 16×16 píxeles en pantalla, sin modificar su contenido.

El código de la ROM utilizado fue provisto por la cátedra.

Funcionamiento del gen_pixeles

Para determinar dónde se permite dibujar, gen_pixeles identifica nuevamente las tres bandas verticales de 128 píxeles correspondientes a cada canal, utilizando los bits más significativos de **pixel_y_i.** Estas se combinan para generar **y_text_on_s**, mientras que **x_text_on_s** permanece activa para aprovechar todo el ancho visible de la pantalla. A partir de ambas señales se obtiene **text_on_s,** que habilita la escritura únicamente dentro de las tres bandas.

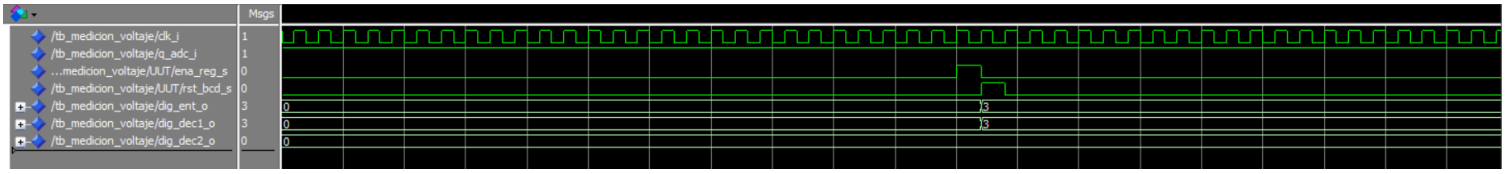
Finalmente, la señal pixel_on_s se obtiene combinando **video_on_i, text_on_s y rom_out_s**, de modo que solo se activa cuando el barrido se encuentra en una región visible y habilitada para texto, y la ROM indica que el píxel pertenece al carácter. A partir de esta señal se generan las salidas RGB, estableciendo los caracteres en magenta y el fondo visible en verde. Las expresiones lógicas utilizadas se presentan en el Anexo A.6.

SIMULACIONES

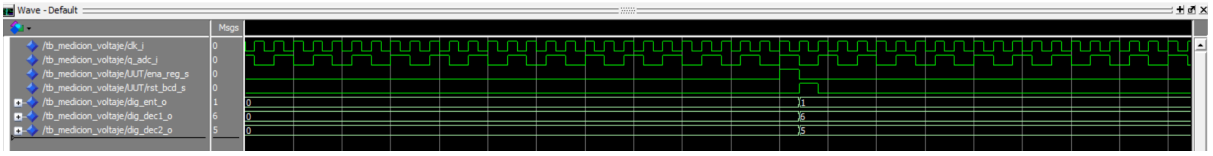
En el siguiente apartado se incluyen algunas simulaciones de los bloques que resultó relevantes corroborar su correcto funcionamiento previo al sintetizado en Quartus.

1. Bloque Medición de Voltaje

Para verificar el funcionamiento del bloque de medición de voltaje se realizaron dos simulaciones utilizando distinta densidades de ‘1’ , en la entrada **q_adc_i**, representando diferentes valores de tensión



En la primera simulación se mantuvo constante **q_adc_i** = ‘1’ durante toda la ventana de medición, simulando la entrada de 3,3 V (el máximo voltaje).
El contador ventana establece , en nuestro caso, una ventana de 33000 ciclos de reloj, durante la cual el contador BCD contabiliza las muestras en la que la entrada se encuentra en nivel lógico alto. En este caso porque la entrada permaneció en ‘1’ constante en la ventana, se contabilizaron 33000 unos. Al completarse la ventana de medición se pone en alto la señal **en_reg_s,** que habilita el registro de la medición obtenida pudienose ver como las salidas **dig_ent_o**, **dig_dec1_o** y **dig_dec2_o** pasan a tomar los valores 3, 3 y 0. En el siguiente flanco de subida del clk la señal **rst_bcd_s** se activa y reinicia el contador BCD.
De esta manera se verificó el correcto conteo de los unos recibido, la selección de los tres dígitos utilizados para representar la tensión y el funcionamiento de los registros, encargados de almacenar la medición por más que el BCD se haya reiniciado.



En la otra simulación se probó lo mismo pero con un 50% de densidad de ‘1’ dándonos así 16500 ‘1’ en la ventada de 33000 ciclos. Obteniendo los valores 1, 6 y 5 representado 1,65 V. Demostrando que no solo responde bien al valor máximo sino también a un valor intermedio en la entrada.

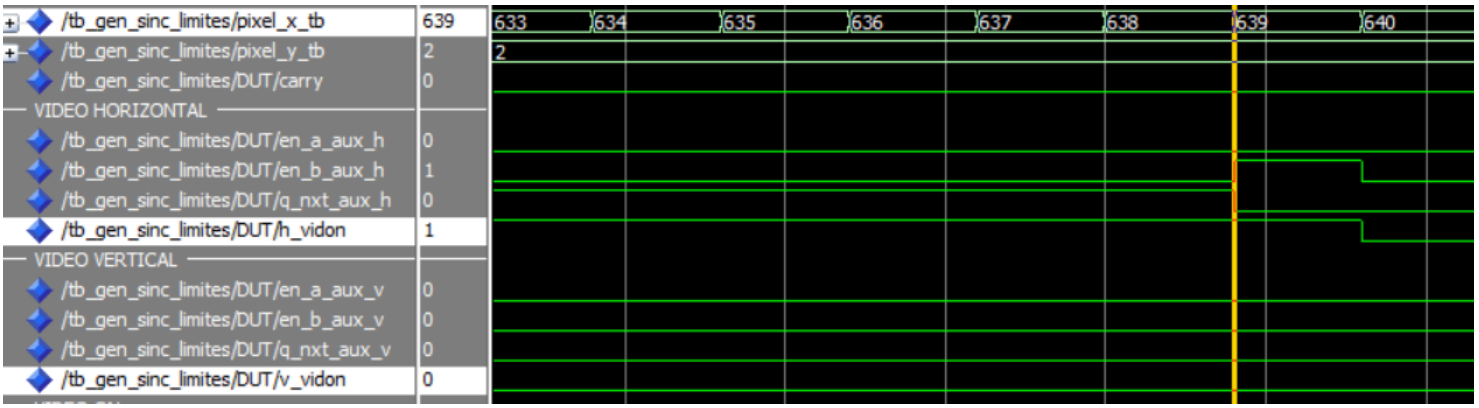
2. Bloque VGA

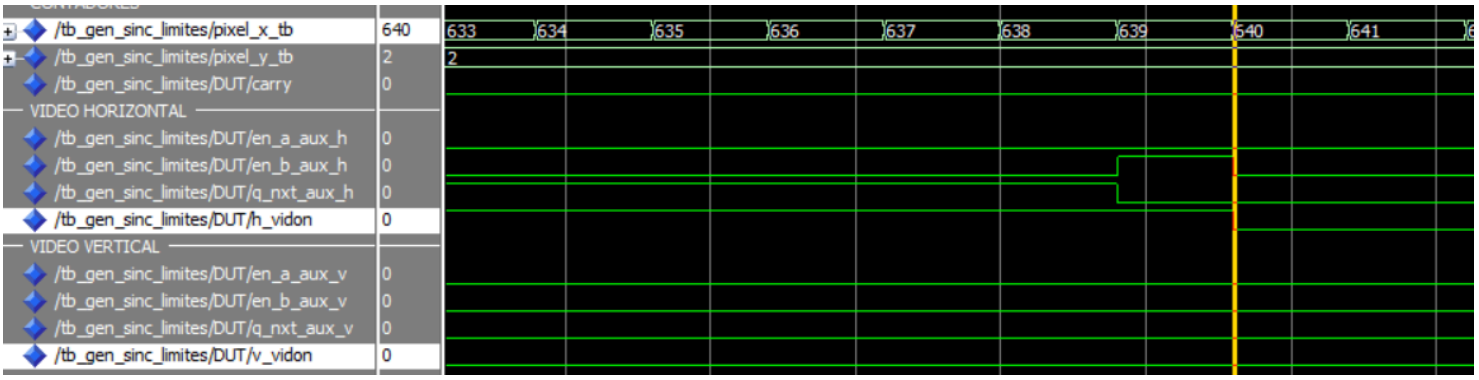
2.1 Generador de sincronismo

Para verificar el funcionamiento de **gen_sinc** se analizaron los límites temporales de las señales de habilitación de video, sincronismo horizontal y vertical, así como la selección de las regiones correspondientes a los tres canales.

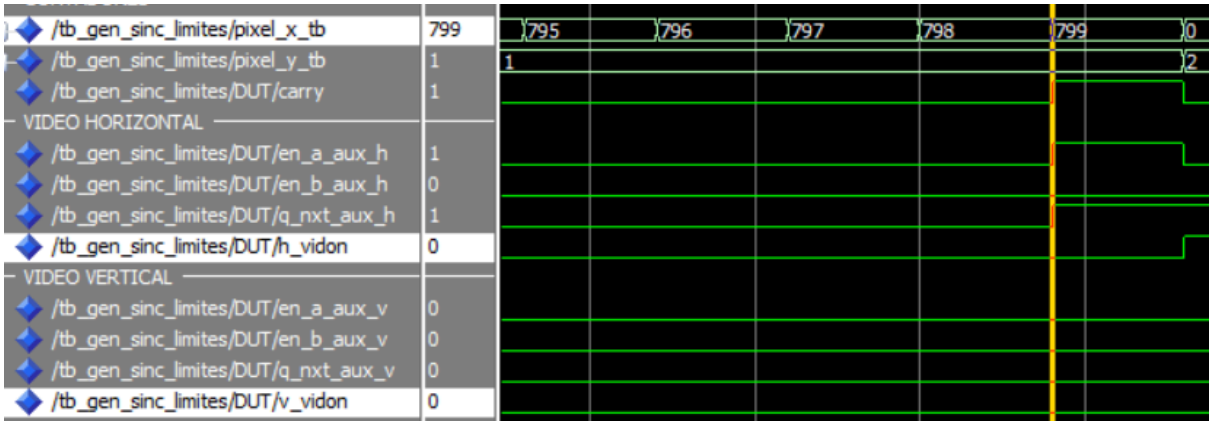
- **Barrido horizontal**

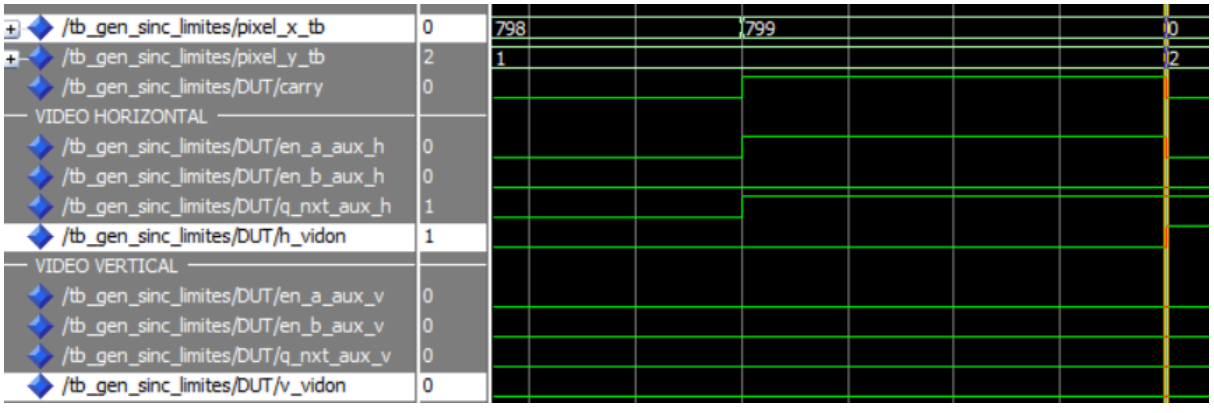
Se verificó que la habilitación horizontal de video coincide exactamente con los 640 píxeles visibles.





Al finalizar la cuenta en 799, **h_vidon** se activa en el mismo flanco en que **pixel_x** vuelve a 0

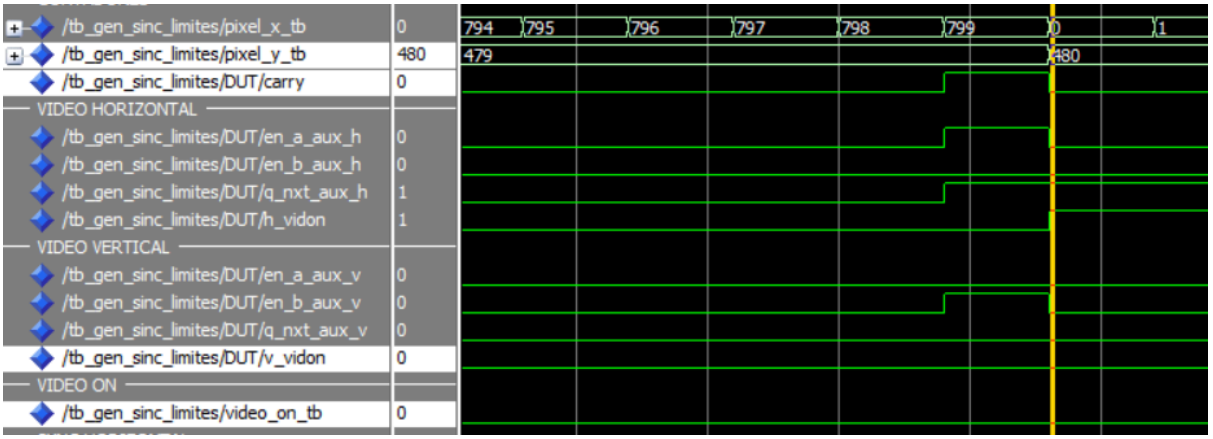




- **Barrido vertical**

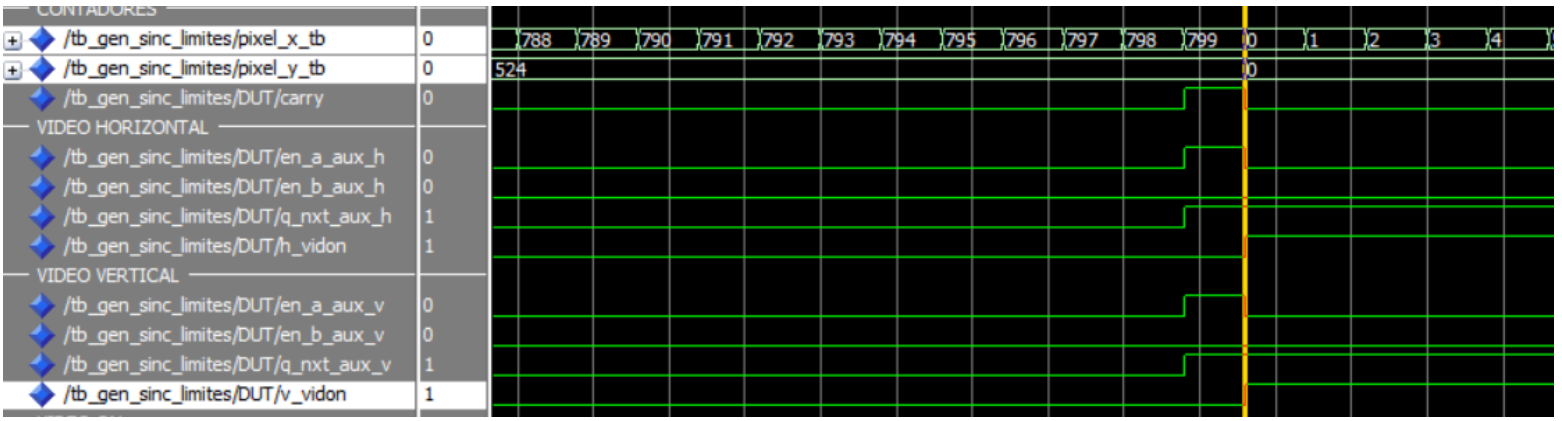
Se verificó que la habilitación vertical permanece activa durante las 480 líneas visibles, desde y=0 hasta y=479, y se desactiva exactamente al comenzar la línea 480.



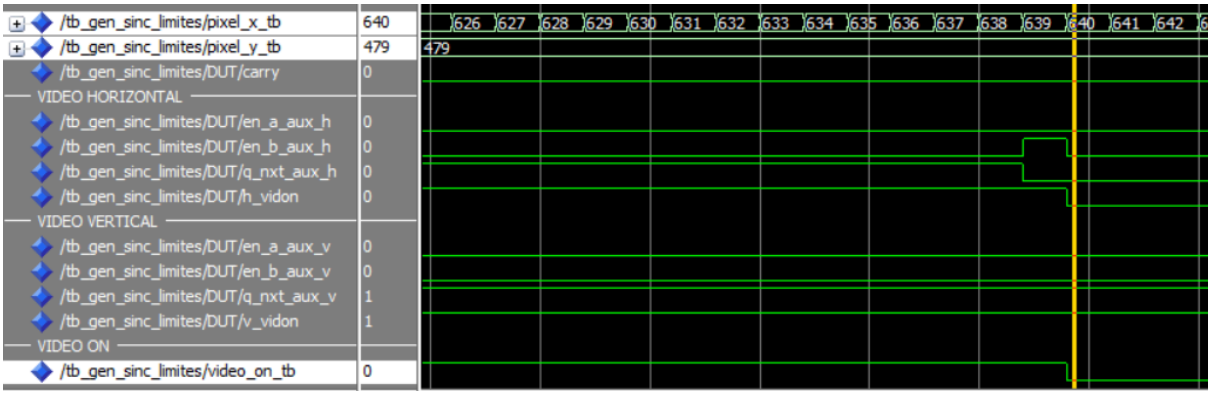


- **Video_on**

Empieza **video_on**



Termina **video_on**

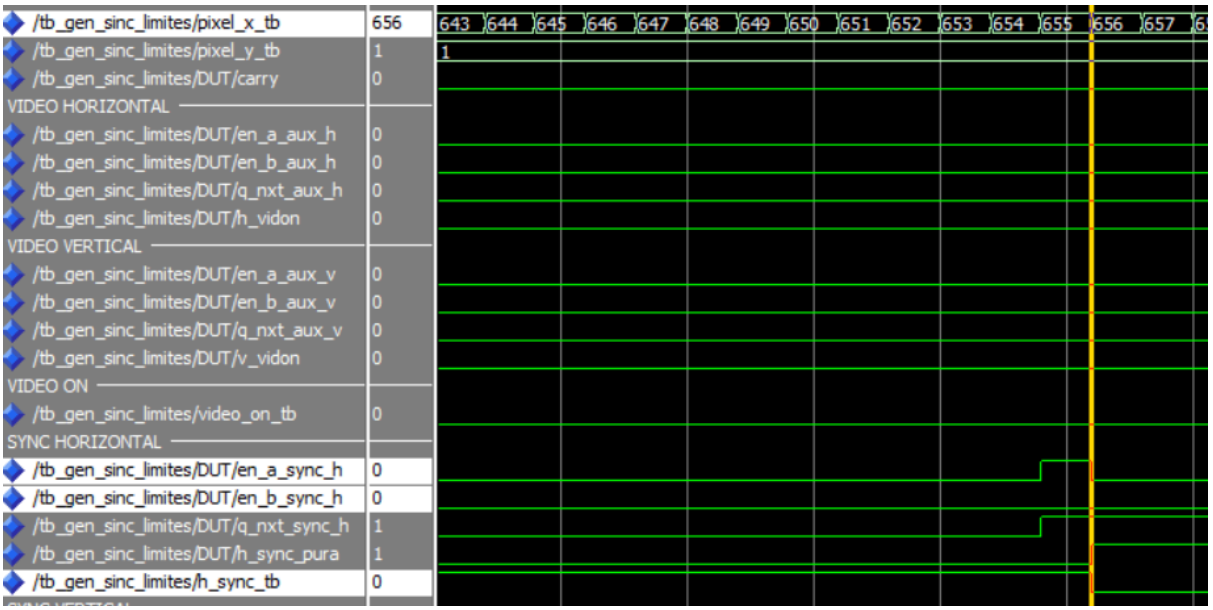


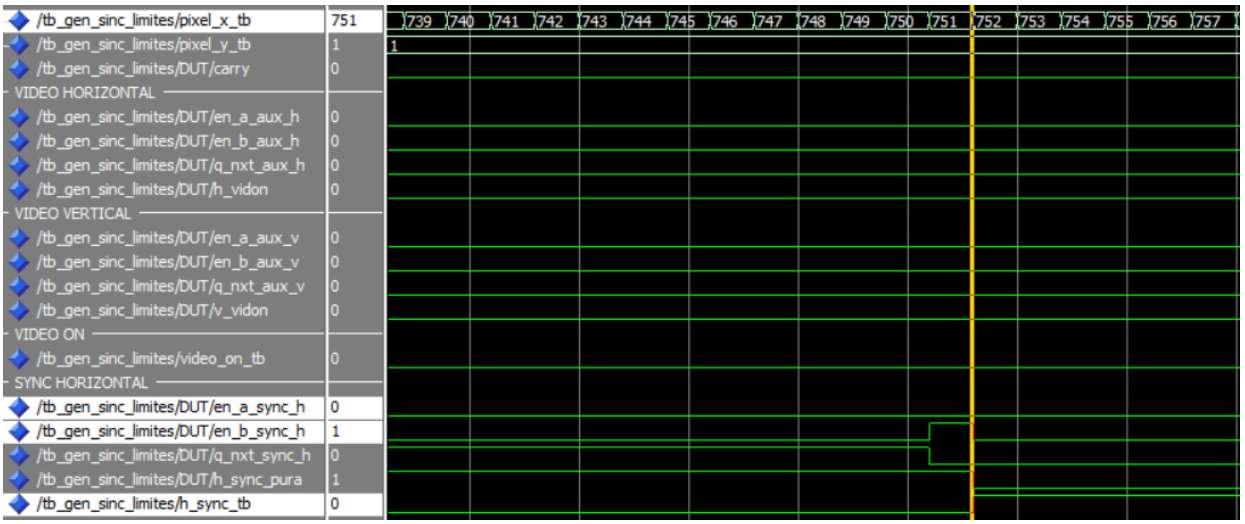
Se comprobó que **h_vidon** permanece activo exactamente entre **pixel_x** = 0 y 639, mientras que **v_vidon** permanece activo durante las líneas **pixel_y** = 0 a 479. De esta forma, **video_on**, obtenido como la combinación de ambas señales, delimita correctamente la región visible de 640 × 480 píxeles

- **Sincronismo horizontal**

Se verificó que **h_sync** presenta su pulso activo en bajo entre pixel_x = 656 y 751,

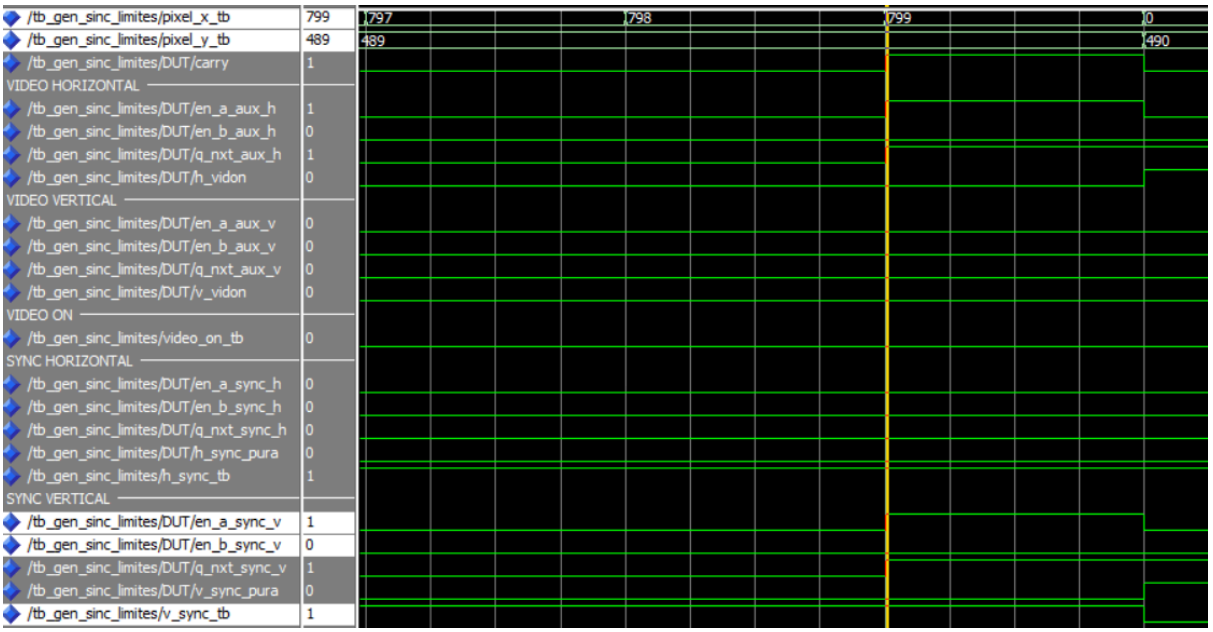


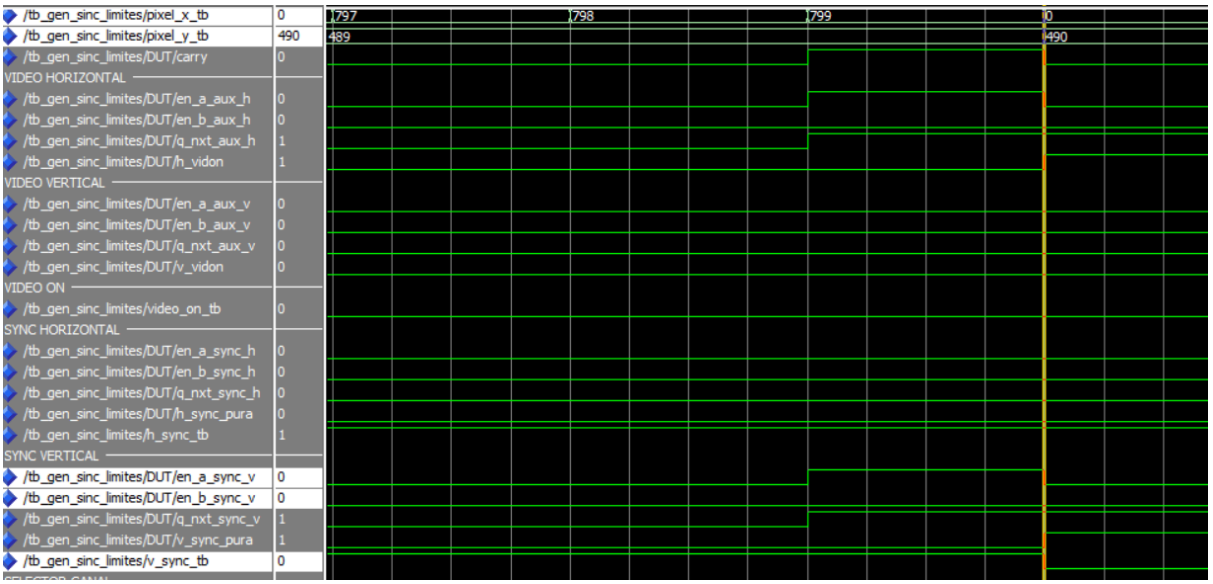

/tb_gen_sinc_limites/pixel_x_tb
656
643 644 645 646 647 648 649 650 651 652 653 654 655 656 657
/tb_gen_sinc_limites/pixel_y_tb
1
1
/tb_gen_sinc_limites/DUT/carry
0
VIDEO HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_aux_h
0
/tb_gen_sinc_limites/DUT/en_b_aux_h
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_h
0
/tb_gen_sinc_limites/DUT/h_vidon
0
VIDEO VERTICAL
/tb_gen_sinc_limites/DUT/en_a_aux_v
0
/tb_gen_sinc_limites/DUT/en_b_aux_v
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_v
0
/tb_gen_sinc_limites/DUT/v_vidon
0
VIDEO ON
/tb_gen_sinc_limites/video_on_tb
0
SYNC HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_sync_h
0
/tb_gen_sinc_limites/DUT/en_b_sync_h
0
/tb_gen_sinc_limites/DUT/q_nxt_sync_h
1
/tb_gen_sinc_limites/DUT/h_sync_pura
1
/tb_gen_sinc_limites/h_sync_tb
0

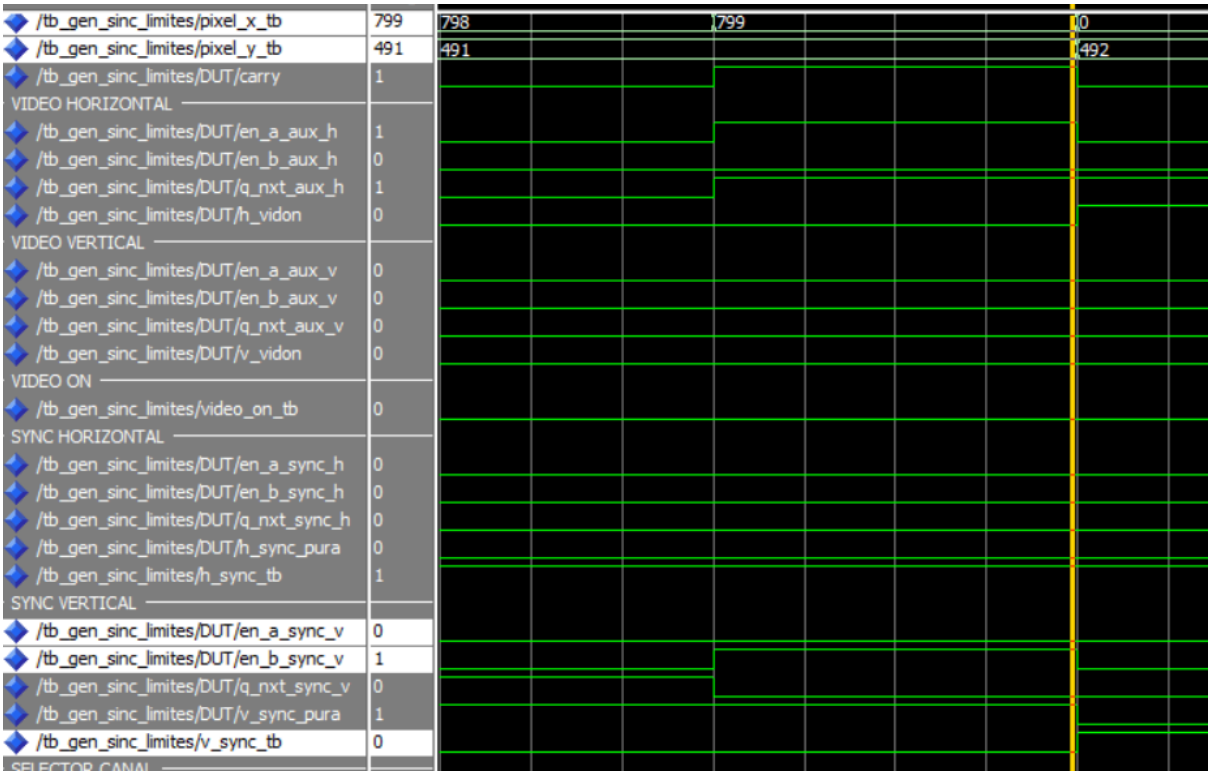

/tb_gen_sinc_limites/pixel_x_tb
751
739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757
/tb_gen_sinc_limites/pixel_y_tb
1
1
/tb_gen_sinc_limites/DUT/carry
0
VIDEO HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_aux_h
0
/tb_gen_sinc_limites/DUT/en_b_aux_h
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_h
0
/tb_gen_sinc_limites/DUT/h_vidon
0
VIDEO VERTICAL
/tb_gen_sinc_limites/DUT/en_a_aux_v
0
/tb_gen_sinc_limites/DUT/en_b_aux_v
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_v
0
/tb_gen_sinc_limites/DUT/v_vidon
0
VIDEO ON
/tb_gen_sinc_limites/video_on_tb
0
SYNC HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_sync_h
0
/tb_gen_sinc_limites/DUT/en_b_sync_h
1
/tb_gen_sinc_limites/DUT/q_nxt_sync_h
0
/tb_gen_sinc_limites/DUT/h_sync_pura
1
/tb_gen_sinc_limites/h_sync_tb
0

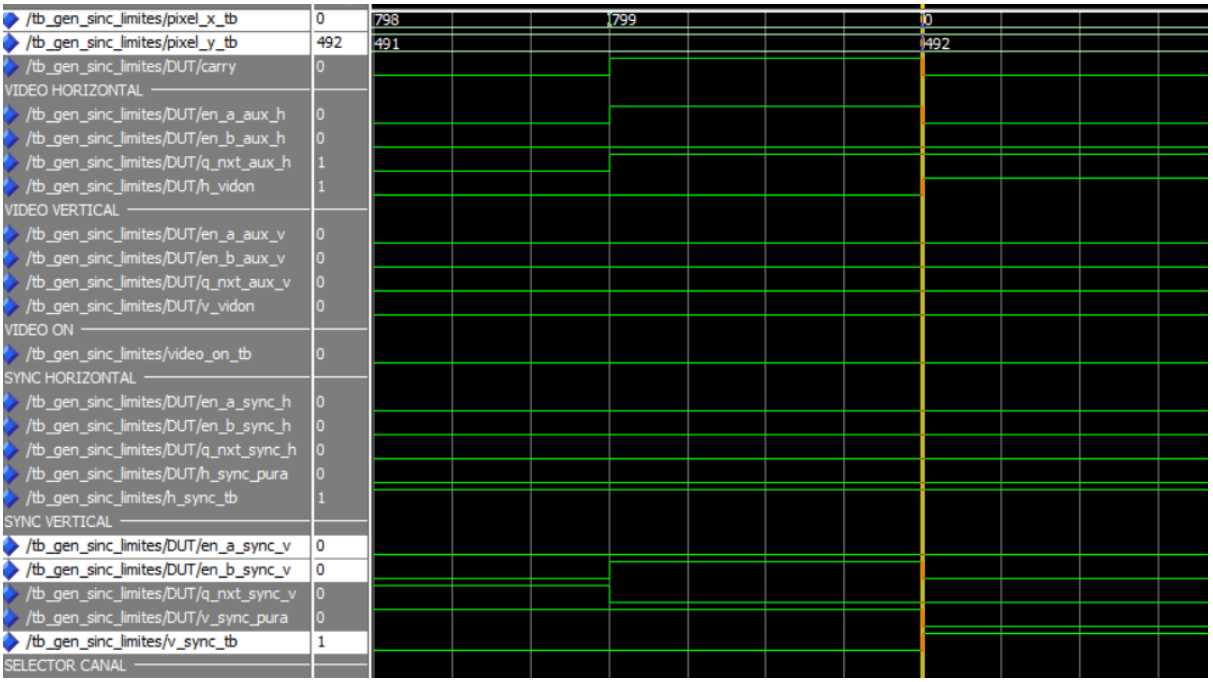
- **Sincronismo vertical**

v_sync se activa exactamente al comienzo de la línea 490 y finaliza al comienzo de la línea 492. Las transiciones verticales se producen únicamente al finalizar una línea, garantizando su correcta alineación temporal.





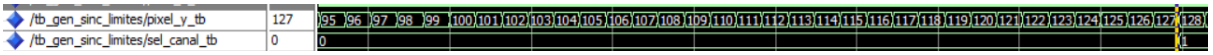

/tb_gen_sinc_limites/pixel_x_tb
799
/tb_gen_sinc_limites/pixel_y_tb
491
/tb_gen_sinc_limites/DUT/carry
1
VIDEO HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_aux_h
1
/tb_gen_sinc_limites/DUT/en_b_aux_h
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_h
1
/tb_gen_sinc_limites/DUT/h_vidon
0
VIDEO VERTICAL
/tb_gen_sinc_limites/DUT/en_a_aux_v
0
/tb_gen_sinc_limites/DUT/en_b_aux_v
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_v
0
/tb_gen_sinc_limites/DUT/v_vidon
0
VIDEO ON
/tb_gen_sinc_limites/video_on_tb
0
SYNC HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_sync_h
0
/tb_gen_sinc_limites/DUT/en_b_sync_h
0
/tb_gen_sinc_limites/DUT/q_nxt_sync_h
0
/tb_gen_sinc_limites/DUT/h_sync_pura
0
/tb_gen_sinc_limites/h_sync_tb
1
SYNC VERTICAL
/tb_gen_sinc_limites/DUT/en_a_sync_v
0
/tb_gen_sinc_limites/DUT/en_b_sync_v
1
/tb_gen_sinc_limites/DUT/q_nxt_sync_v
0
/tb_gen_sinc_limites/DUT/v_sync_pura
1
/tb_gen_sinc_limites/v_sync_tb
0
798
799
0
491
492


/tb_gen_sinc_limites/pixel_x_tb
0
/tb_gen_sinc_limites/pixel_y_tb
492
/tb_gen_sinc_limites/DUT/carry
0
VIDEO HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_aux_h
0
/tb_gen_sinc_limites/DUT/en_b_aux_h
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_h
1
/tb_gen_sinc_limites/DUT/h_vidon
1
VIDEO VERTICAL
/tb_gen_sinc_limites/DUT/en_a_aux_v
0
/tb_gen_sinc_limites/DUT/en_b_aux_v
0
/tb_gen_sinc_limites/DUT/q_nxt_aux_v
0
/tb_gen_sinc_limites/DUT/v_vidon
0
VIDEO ON
/tb_gen_sinc_limites/video_on_tb
0
SYNC HORIZONTAL
/tb_gen_sinc_limites/DUT/en_a_sync_h
0
/tb_gen_sinc_limites/DUT/en_b_sync_h
0
/tb_gen_sinc_limites/DUT/q_nxt_sync_h
0
/tb_gen_sinc_limites/DUT/h_sync_pura
0
/tb_gen_sinc_limites/h_sync_tb
1
SYNC VERTICAL
/tb_gen_sinc_limites/DUT/en_a_sync_v
0
/tb_gen_sinc_limites/DUT/en_b_sync_v
0
/tb_gen_sinc_limites/DUT/q_nxt_sync_v
0
/tb_gen_sinc_limites/DUT/v_sync_pura
0
/tb_gen_sinc_limites/v_sync_tb
1
SELECTOR CANAL
798
799
0
491
492

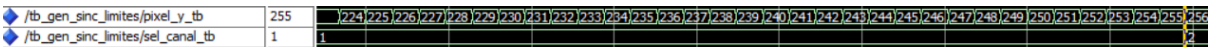
- **Bandas**

Finalmente, **sel_canal_s** toma los valores 0, 1 y 2 en las regiones verticales correspondientes a cada uno de los tres canales de medición, verificando la correcta generación de la señal utilizada posteriormente por los multiplexores del bloque de procesamiento.

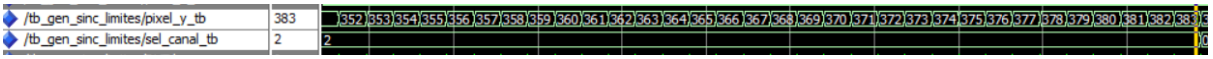
banda 0



banda 1

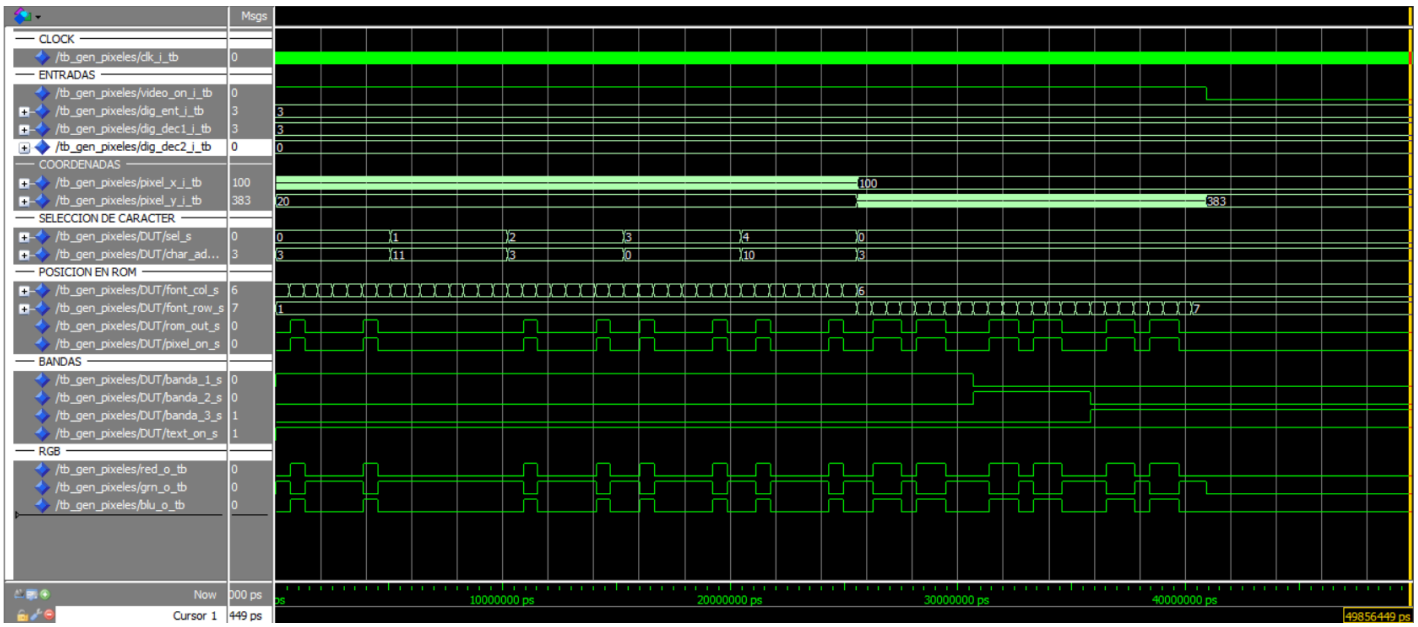


banda 2

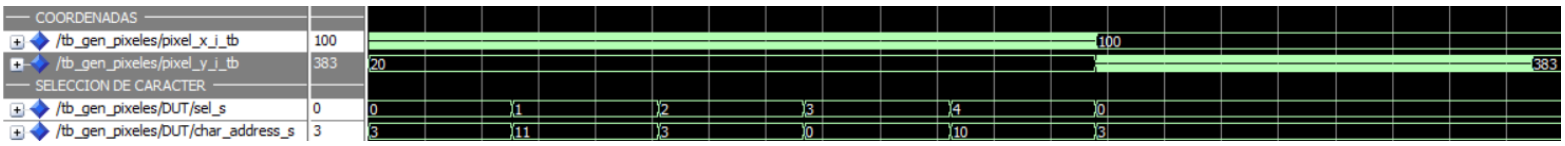


2.2 Generador de píxeles

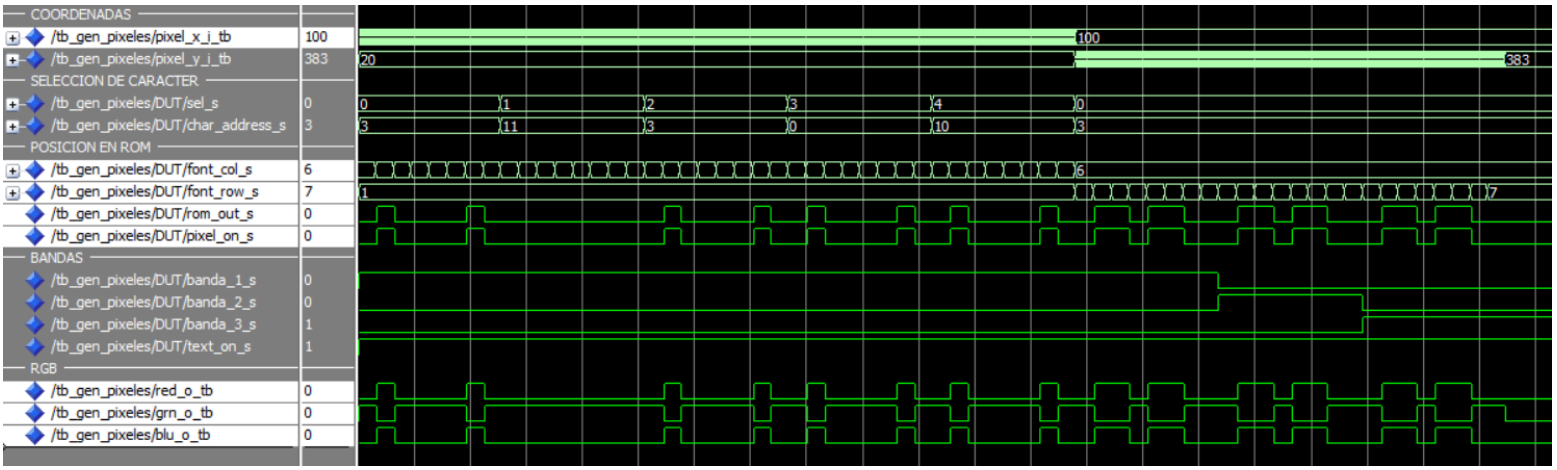
Para verificar el funcionamiento de **gen_pixeles** se fijaron los dígitos 3, 3 y 0, correspondientes a una medición de 3,30 V, y se realizó un barrido de la coordenada horizontal.



Se comprobó que **sel_s** divide los 640 píxeles visibles en cinco regiones de 128 píxeles y que **char_address_s** selecciona sucesivamente los caracteres 3, punto, 3, 0 y V. Dentro de cada carácter, **font_col_s** y **font_row_s** recorren la ROM de 8 × 8, repitiendo cada píxel 16 veces para obtener caracteres de 128 × 128 píxeles.

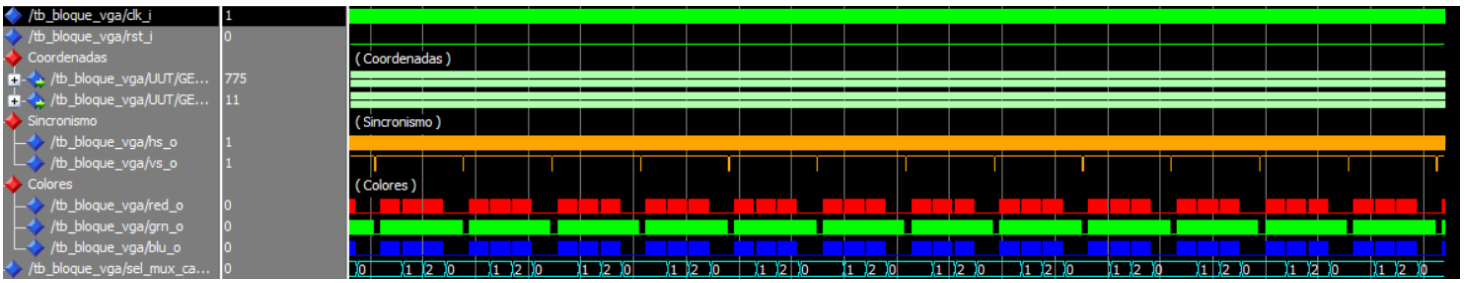


Finalmente, se verificó que **rom_out_s,** obtenida a partir de la fila y columna seleccionadas de la ROM, determina si el píxel actual pertenece al carácter o al fondo. Cuando el píxel (pixel_on) pertenece al carácter se activan red_o y blu_o, obteniéndose color magenta; en caso contrario se activa grn_o, generando el fondo verde. De esta manera se comprobó la correcta lectura de la ROM y su conversión en las señales RGB de salida.

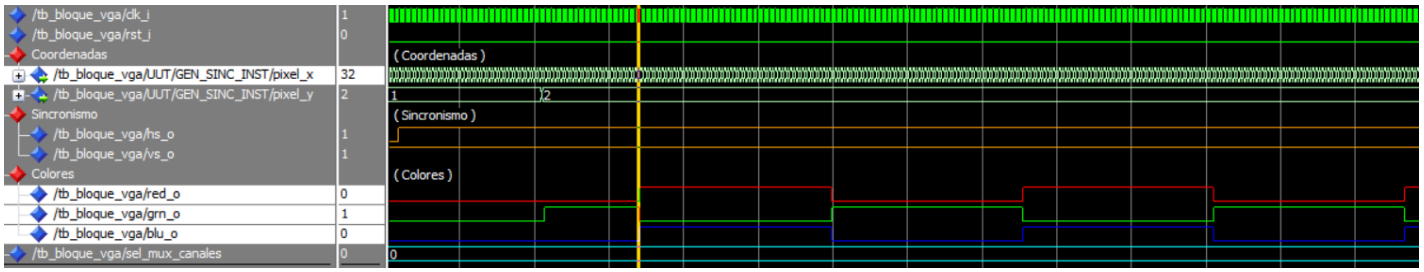


3. VGA

Por último, se simuló el bloque VGA completo, permitiendo verificar conjuntamente el funcionamiento del generador de sincronismo y del generador de píxeles.



En la simulación se observa la generación periódica de las señales **h_sync y v_sync**, junto con el avance de las coordenadas horizontal y vertical. También se verificó que **sel_mux_canales** toma sucesivamente los valores 0, 1 y 2 según la coordenada vertical, permitiendo seleccionar el canal correspondiente a cada franja de la pantalla y repitiendo esta secuencia en cada frame.



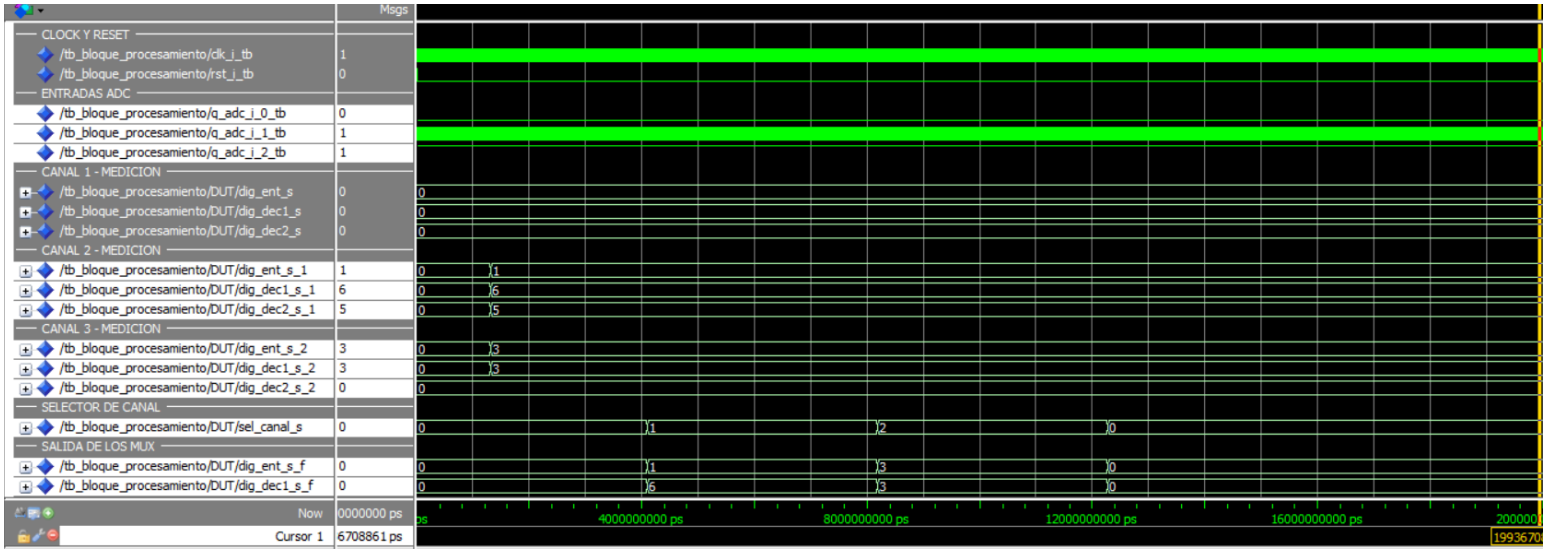
Por otra parte, las señales RGB presentan color verde cuando el píxel correspondiente al carácter se encuentra apagado, mientras que r**ed_o** y **blu_o** se activan simultáneamente cuando debe dibujarse un píxel del carácter, generando color magenta. De esta manera se comprobó la correcta integración entre el generador de sincronismo, la selección de canales y el generador de píxeles dentro del bloque VGA.

4. Bloque de Procesamiento

Para verificar el funcionamiento del bloque procesamiento se aplicaron tres señales diferentes a las entradas provenientes de los ADC. El canal 0 representa 0V, el canal 1 se generó con una densidad de 1 del 50% representando 1,65 V y el canal 3 con el 100% de densidad de unos correspondientes a 3,30 V.

Se vio que los 3 bloques de medición entregan los dígitos respectivamente, validando el procesamiento independiente de cada canal

Y finalmente, se observó que , según el valor de **sel_canal_s**, los MUX seleccionan correctamente uno de los tres resultados y lo transfieren a las señales finales que ingresan a la VGA.



RESUMEN UTILIZACIÓN DE RECURSOS QUARTUS II

Flow Status	Successful - Fri Oct 02 17:24:36 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	top_level
Top-level Entity Name	top_level
Family	Cyclone II
Device	EP2C5AT144A7
Timing Models	Final
Total logic elements	333 / 4,608 (7 %)
Total combinational functions	333 / 4,608 (7 %)
Dedicated logic registers	141 / 4,608 (3 %)
Total registers	141
Total pins	13 / 89 (15 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)

Luego de la síntesis del diseño completo en Quartus II para la FPGA Cyclone II EP2C5AT144A7, se obtuvo una utilización de 333 de los 4608 elementos lógicos disponibles (7 %), 141 registros dedicados (3 %) y 13 de los 89 pines disponibles (15 %).

ERRORES Y ASPECTOS A MEJORAR

Durante la implementación se detectaron dos errores principales de código, ambos relacionados con señales de control del sistema.

El primer error estuvo asociado a la polaridad del reset. En el diseño original, los bloques internos estaban implementados considerando un reset activo en alto. Sin embargo, al probar el circuito en la placa se observó que la imagen VGA aparecía únicamente mientras se mantenía presionado el botón de reset. Esto indicó que el reset físico de la placa era activo en bajo. Por lo tanto, cuando el botón no estaba presionado, la señal mantenía al sistema reseteado permanentemente. La solución se realizó en el top_level, generando una señal invertida: rst_inv <= not rst_i

A partir de esa modificación, rst_inv se utilizó como reset interno para los bloques del sistema, logrando que el circuito funcione normalmente cuando el botón no está presionado y se resetee únicamente al accionarlo.

El segundo error apareció al escalar los caracteres de la VGA. Inicialmente, los caracteres de la ROM se trabajaban como símbolos de 8 × 8 píxeles. Luego, para que ocuparan correctamente el ancho de la pantalla, se escalaron a 128 × 128 píxeles modificando el gen_pixeles. Sin embargo, la lógica de selección de canal no se actualizó al mismo criterio. En particular, la señal sel_canal_s, generada a partir de las bandas verticales, seguía respondiendo a las divisiones anteriores. Como consecuencia, los multiplexores del bloque_procesamiento permanecían seleccionando casi siempre el canal 1, por lo que las tres filas mostraban el valor correspondiente a ese canal.

La corrección consistió en unificar el criterio de bandas entre el gen_pixeles y el bloque que genera sel_canal_s. Se definieron tres regiones verticales de 128 píxeles: y = 0..127 para el canal 1, y = 128..255 para el canal 2 y y = 256..383 para el canal 3. De esta manera, la selectora toma los valores 00, 01 y 10 según la banda recorrida, permitiendo que los tres multiplexores seleccionen correctamente los dígitos del canal correspondiente.

Como aspecto a mejorar, el diseño podría optimizarse reduciendo la cantidad de bloques repetidos. En lugar de utilizar tres bloques medicion_voltaje en paralelo, podría multiplexarse la salida de los ADC y emplearse un único bloque de medición. De esta forma, se obtendría una arquitectura más económica en recursos lógicos.

ANEXO

A.1. Diseño lógico del contador decimal (cont_9)

Diagrama de estados, tabla de transición y mapas de Karnaugh utilizados para obtener las ecuaciones de próximo estado y la salida de acarreo.

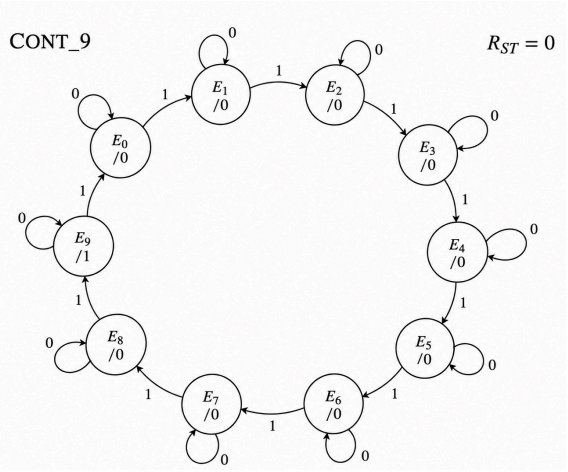


Tabla de transición de estados

Dec.	Estado $Q_3Q_2Q_1Q_0$	Próximo $D_3D_2D_1D_0$	cout
0	0000	0001	0
1	0001	0010	0
2	0010	0011	0
3	0011	0100	0
4	0100	0101	0
5	0101	0110	0
6	0110	0111	0
7	0111	1000	0
8	1000	1001	0
9	1001	0000	1

Estados 1010 a 1111: condiciones indiferentes (X).

Mapa de Karnaugh del acarreo (cout)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	X	X	X	X
10	0	1	X	X

$$C_{OUT} = Q_3 * Q_0$$

Mapa de Karnaugh de D_0

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

$$D_0 = \overline{Q_0}$$

Mapa de Karnaugh de D_1

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	X	X	X	X
10	0	0	X	X

$$D_1 = Q_1 * \overline{Q_2} + \overline{Q_3} * \overline{Q_1} * Q_0$$

Mapa de Karnaugh de D_2

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	0	1	0
01	1	1	0	1
11	X	X	X	X
10	0	0	X	X

$$D_2 = Q_2 * \overline{Q_1} + \overline{Q_2} * Q_0 * Q_1 + Q_2 * \overline{Q_0}$$

Mapa de Karnaugh de D_3

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	X	X	X	X
10	1	0	X	X

$$D_3 = Q_2 * Q_0 * Q_1 + Q_3 * \overline{Q_0}$$

A.2. Implementación de la lógica de control de video y sincronismo

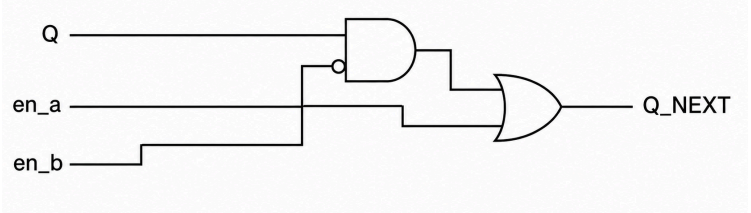


Figura A.2.1. *Esquema lógico del bloque logic_int, utilizado para calcular el próximo estado de las máquinas de Moore implementadas en la generación de video_on, h_sync y v_sync.*

$$Q_{next} = Q * \overline{en_b} + en_a$$

Las entradas en_a y en_b de `logic_int` permiten pasar a los estados con salida 1 y 0, respectivamente, según los valores de los contadores indicados en los siguientes diagramas.

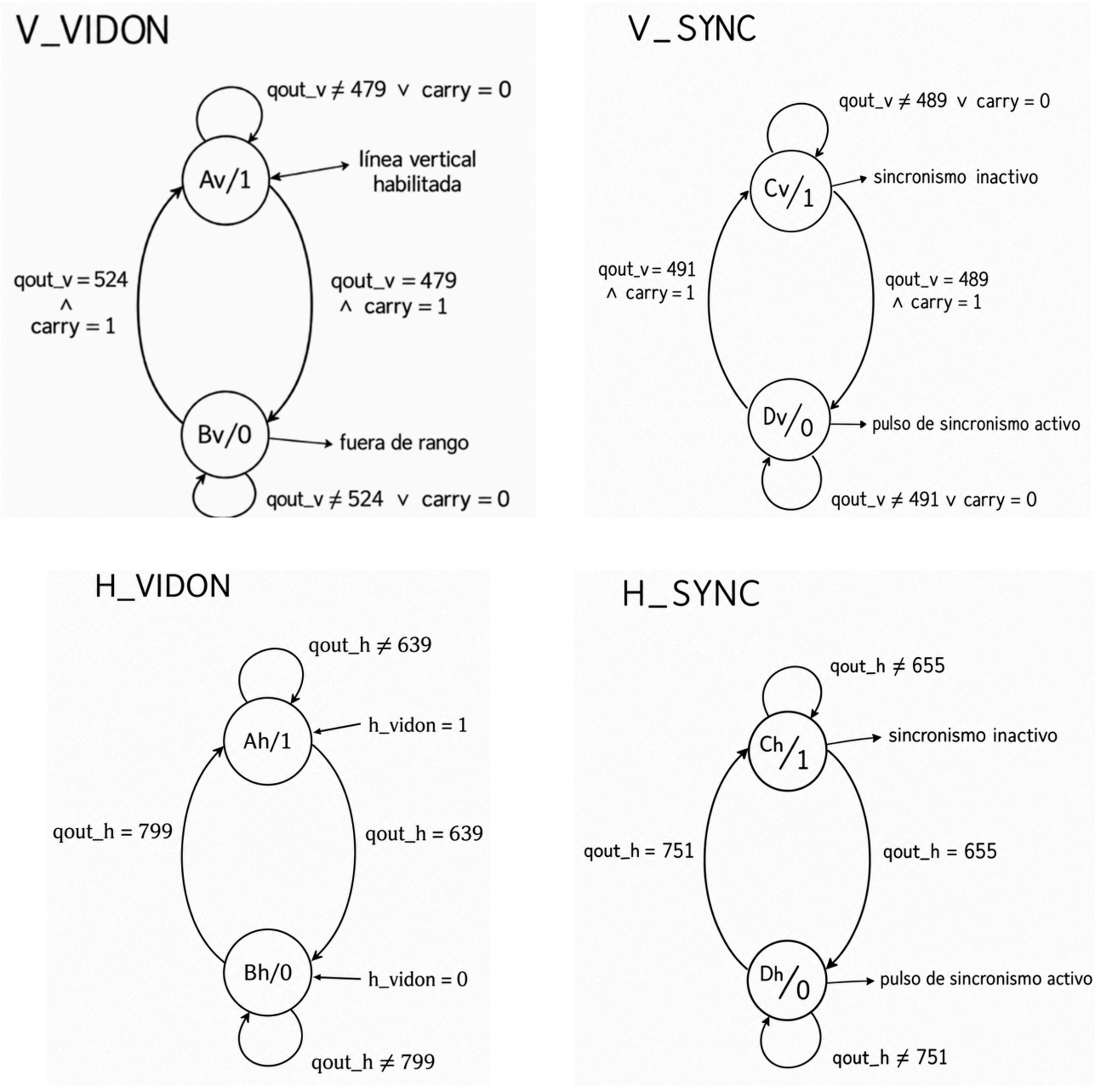


Figura A.2.2. *Diagramas de estados de h_vidon, v_vidon, h_sync y v_sync.*

A.3 Selección de bandas verticales (gen_sinc)

Tabla A.3.1 *Tabla de verdad de las bandas verticales y la señal de selección de canal.*

qout_v(9)	***qout_v(8)***	***qout_v(7)***	***banda_1_s***	***banda_2_s***	***banda_3_s***	***sel_canal_s***
0	*0*	*0*	*1*	*0*	*0*	*00*
0	*0*	*1*	*0*	*1*	*0*	*01*
0	*1*	*0*	*0*	*0*	*1*	*10*
0	*1*	*1*	*0*	*0*	*0*	*00*
1	*0*	*0*	*0*	*0*	*0*	*00*
1	*0*	*1*	*0*	*0*	*0*	*00*
1	*1*	*0*	*0*	*0*	*0*	*00*
1	*1*	*1*	*0*	*0*	*0*	*00*

Expresiones lógicas:

$$Banda\ 1 = \overline{Q_9} * \overline{Q_8} * \overline{Q_7} \qquad Banda\ 2 = \overline{Q_9} * \overline{Q_8} * Q_7 \qquad Banda\ 3 = \overline{Q_9} * Q_8 * \overline{Q_7}$$

Siendo Q_7, Q_8, Q_9 *los bits 9, 8 y 7 del contador vertical* Q_{out_v}

A partir de las bandas verticales se generan los dos bits de sel_canal_s, que controlan el multiplexor de tres canales explicado en el Anexo A.4.

$$Sel_canal_s(1) = Banda_3 \qquad Sel_canal_s(0) = Banda_2$$

A.4. Diseño del multiplexor de canales (mux3a1_4b)

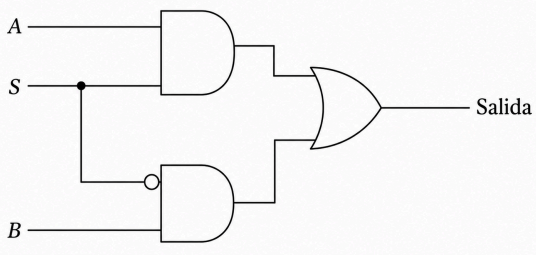


***Figura A.4.1**. Esquema lógico del multiplexor 2 a 1 de 1 bit (mux2a1).*

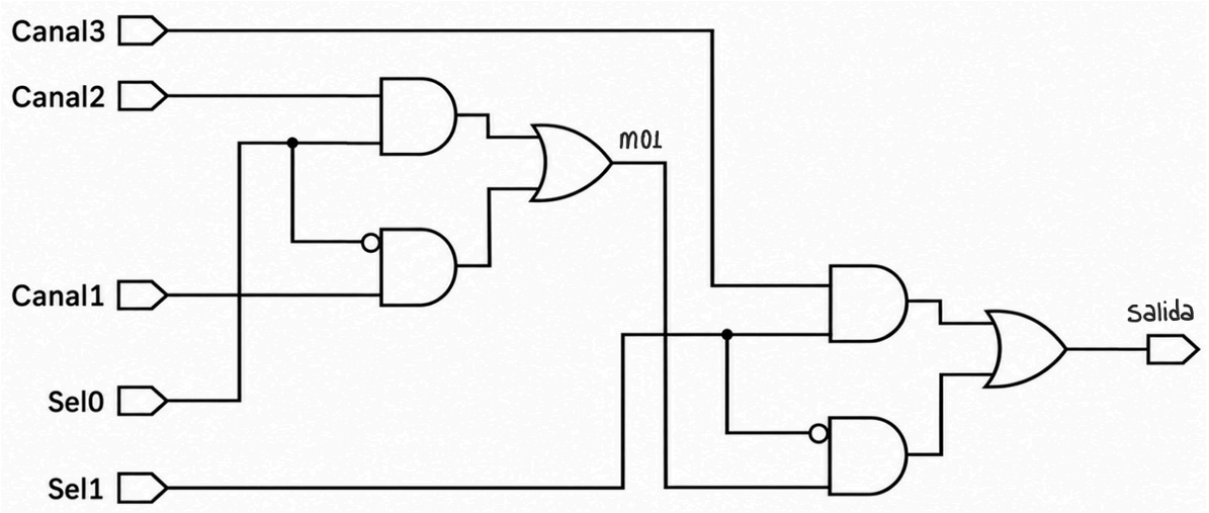


***Figura A.4.2.** Implementación del multiplexor 3 a 1 de 4 bits (mux3a1_4b) mediante dos multiplexores 2 a 1 conectados en cascada.*

***Tabla A.4.1.** Tabla de selección de canales del mux3a1_4b, indicando la salida intermedia m01 y la salida final sal*

sel(1)	sel(0)	m01	Salida sal
0	0	Canal 1	Canal 1
0	1	Canal 2	Canal 2
1	0	Canal 1	Canal 3
1	1	Canal 2	Canal 3

A.5 Diseño del multiplexor de caracteres (mux_char_address)

Tabla A.5.1. *Tabla de selección de caracteres del mux_char_address según la combinación de los bits de sel_i.*

sel_i = s2 s1 s0	Entrada seleccionada	Descripción
000	e0	Dígito entero
001	e1	Punto
010	e2	Primer decimal
011	e3	Segundo decimal
100	e4	Letra V
101	e5	Espacio
110	e6	Espacio
111	e7	Espacio

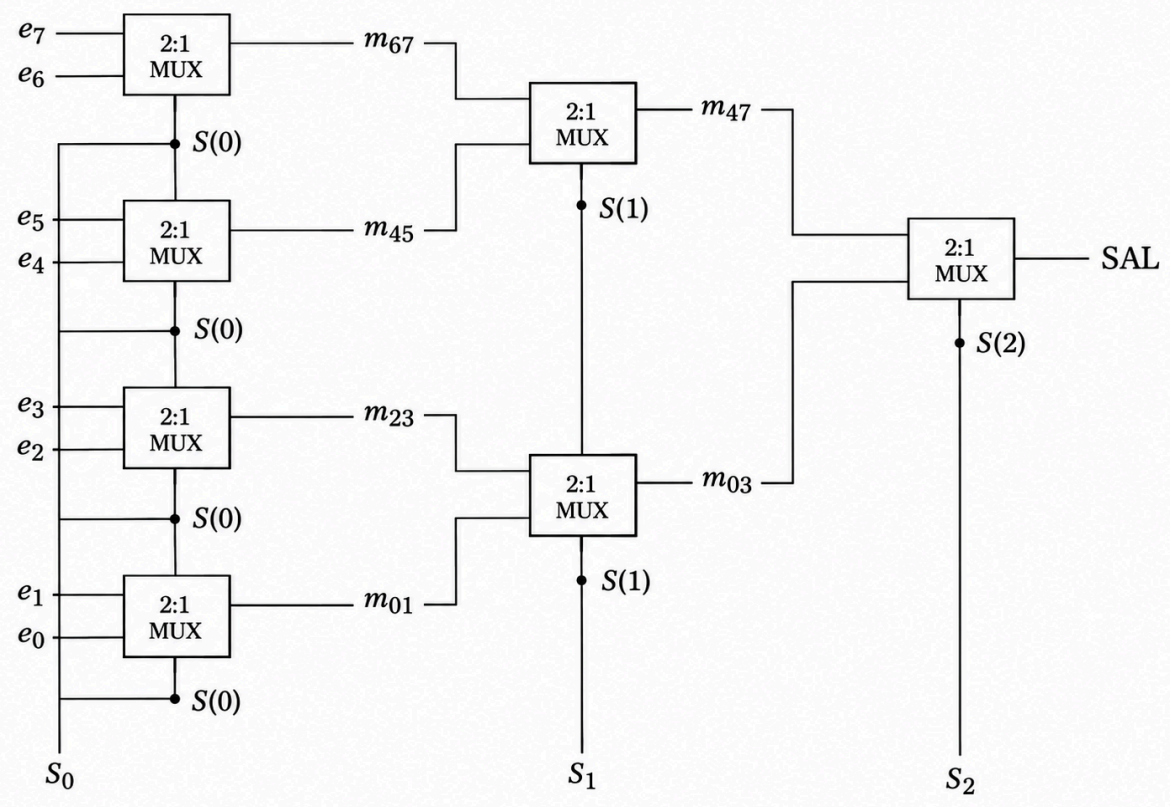


Figura A.5.1. *Implementación del multiplexor 8 a 1 de 4 bits (mux8a1_4b) mediante multiplexores 2 a 1 conectados en cascada.*

A.6. Lógica de generación de colores RGB (gen_pixeles)

Tabla A.6.1. *Combinaciones de las salidas RGB y colores correspondientes.*

R	G	B	Color
0	0	0	Negro
0	0	1	Azul
0	1	0	Verde
0	1	1	Cian
1	0	0	Rojo
1	0	1	Magenta
1	1	0	Amarillo
1	1	1	Blanco

Tabla A.5.1. *Tabla de verdad de las salidas RGB según la habilitación de video, la región de texto y la salida de la ROM.*

video_on_i	text_on_s	rom_out_s	pixel_on_s	RGB	Color
0	X	X	0	000	Negro
1	0	X	0	010	Verde
1	1	0	0	010	Verde
1	1	1	1	101	Magenta

Expresiones lógicas:

$$Pixel_on_s = video_on_i * text_on_s * rom_out_s$$

$$red_o = Pixel_on_s$$

$$grn_o = video_on_i * \overline{Pixel_on_s}$$

$$blu_o = Pixel_on_s$$